RFC — BaaS PME: conta, cobrança e liquidez

Time / data / versão	Cairê Belo · Pedro Campanha · Luka Djukic · 10/10/2026 · 3.4 · RFC integral

Contextualização

Entendendo o problema

Uma PME precisa cobrar, movimentar o caixa e antecipar recebíveis com segurança. Duplicar dinheiro ou lastro e perder o histórico comprometem a confiança no saldo. A solução reúne essas operações com identidade, personalização e auditoria. Empréstimo, baixa automática, estorno, múltiplas moedas e SLO ficam fora do escopo. Apresentamos **garantias verificadas e limitações conhecidas**: o líquido positivo está implementado, mas as demais fronteiras de P0.4 seguem pendentes.

Explicando a solução de forma macro

Um serviço HTTP modular e um worker compartilham o PostgreSQL. O fluxo passa por middleware, schema, resource, controller, repository e model/DTO; conectores e MockServer isolam o I/O. Travas, commit e idempotência protegem o dinheiro, enquanto auditoria e outbox registram os fatos. Há exceções à separação de camadas: preço e risco nos repositories, além de SQL na auditoria. Não há gateway nem frota de microserviços.

- **Microserviço por operação** fragmentaria a transação ACID, mas serviria a domínios e deploys independentes. **Lock otimista** exigiria repetir decisões sob disputa, mas seria útil com baixa contenção.
- **Float** perde precisão; **NUMERIC** seria exato, mas centavos simplificam o contrato. Frações de centavo justificariam NUMERIC. **Soft delete** oculta o histórico e serviria a dados descartáveis, não a fatos financeiros.

Implementação

Rotas

O token interno é obrigatório, exceto na raiz e na saúde. Nas contas, o JWT, quando presente, valida PME e papel; propostas exigem OWNER. Bootstrap, políticas diretas, auditoria e métricas não têm RBAC próprio. O erro QIT usa `{title, description, translation, code}`, sem seguir integralmente a RFC 9457. Os erros globais são 403 para token, 401 para JWT, 403 para papel e 503 para timeout SQL. O replay combina UNIQUE(conta,escopo,chave), hash e resposta; os contratos estão em DECISOES.

Método	Caminho	Função / idempotência	Entrada	Saídas (status e quando)
GET	`/`	Identifica	—	200
GET	`/health_check`	Liveness	—	204
POST	`/customer`	Cadastra	CPF/CNPJ, e-mail	201; 400 schema; 409 duplicado; 422 documento
GET	`/customer/{customer_key}`	Consulta	UUID	200; 404 ausente
POST	`/account`	Abre conta	PME	201; 400 schema; 404 PME
GET	`/account/{account_key}`	Consulta	UUID	200; 404 conta
PUT	`/account/{account_key}/block`	Bloqueia	OWNER*	200; 404 conta; 409 estado
PUT	`/account/{account_key}/cancel`	Cancela	OWNER*	200; 404 conta; 409 estado
POST	`/account/{account_key}/transaction`	Movimenta/replay	tipo, valor, destino, chave	201; 400 contrato; 404 conta; 409 estado/chave/produto; 422 saldo/limite/destino; 503 retry
GET	`/account/{account_key}/transaction/{transaction_key}`	Lançamento	UUIDs	200; 404 ausente/alheio
GET	`/account/{account_key}/transactions`	Extrato	página/limite/tipo	200; 400 consulta; 404 conta
POST	`/account/{account_key}/billing-plan`	Lote 1	base, data	201; 400 schema; 404 conta; 409 estado/produto; 422 data/tarifa; 502 emissor
GET	`/account/{account_key}/billing-plan/{plan_key}`	Plano	UUIDs	200; 404 ausente/alheio
POST	`/account/{account_key}/billing-plan/{plan_key}/adjustment`	Lote 2	índice	201; 400 schema; 404 plano; 409 lote/estado/produto; 422 tarifa; 502 conector
POST	`/account/{account_key}/credit-advance`	Antecipa/replay	boletos, chave	201; 400 contrato; 404 lastro; 409 estado/chave/lastro/produto; 422 risco/líquido; 503 retry
POST	`/pricing-policy`	Preço direto	PME, tarifa	201; 400 schema; 404 PME
POST	`/risk-policy`	Risco direto	PME, limites	201; 400 schema; 404 PME
POST	`/account/{account_key}/quote`	Prévia	produto, valor/lastro	201; 400 schema; 404 conta/lastro; 409 produto/lastro; 422 risco/líquido de antecipação
POST	`/policy-change-request`	DRAFT	política, OWNER	201; 400 schema; 401 JWT; 403 vínculo; 404 PME
PUT	`/policy-change-request/{request_key}/submit`	Submete	criador	200; 401 JWT; 403 vínculo; 404 proposta; 409 ator/estado
PUT	`/policy-change-request/{request_key}/approve`	Aprova	outro OWNER	200; 401 JWT; 403 vínculo; 404 proposta; 409 ator/estado
POST	`/user`	Usuário	PME, senha, papel	201; 400 schema/senha; 404 PME; 409 e-mail
POST	`/auth/login`	Nova sessão	credenciais	201; 400 schema; 401 credenciais
POST	`/auth/refresh`	Rotaciona	refresh	201; 400 schema; 401 inválido/usado
POST	`/auth/logout`	Revoga	JWT	204; 401 sessão
GET	`/audit-events`	Cadeia	token	200; 403 token
GET	`/audit-events/checkpoint`	Ponta	token	200; 403 token
GET	`/metrics`	Métricas	token	200; 403 token

*OWNER é exigido quando há JWT. Escritas sem “replay” criam nova entidade ou versão. Repetir mudanças de estado, lote 2 ou proposta gera conflito; refresh é de uso único e logout recusa sessão revogada.

Banco de Dados (Somente diagrama)

DER 1/2 — contas, ledger, cobrança e lastro

PK: chave interna | UK: única | FK: vínculo | relação aponta para este cartão; UUIDs são públicos

CUSTOMER

```
int id PK
char36 customer_key UK
varchar document_number UK
varchar email UK
```

TRANSACTION

```
int id PK
char36 transaction_key UK
int account_id FK
char36 operation_key
bigint amount [com sinal; CHECK != 0]
bigint balance_after
timestamp created_at
```

```
ACCOUNT ||--o{ lanca
ACCOUNT |o--o{ contraparte
PRICING_SNAPSHOT |o--o{ tarifa
RISK_POLICY_SNAPSHOT |o--o{ decisao
```

BANK_SLIP

```
int id PK
char36 slip_key UK [API
bank_slip_key]
int billing_plan_id FK
int credit_advance_id FK
int installment_number [UNIQUE
plano+parcela]
int batch_number
numeric adjustment_rate
bigint amount
date due_date
```

```
BILLING_PLAN ||--o{ gera
CREDIT_ADVANCE |o--o{ lastreia
BANK_SLIP_STATUS ||--o{ estado_atual
PRICING_SNAPSHOT |o--o{ emissao
```

QUOTE

```
int id PK
char36 quote_key UK
int account_id FK
varchar operation
char64 request_hash
bigint gross_amount
bigint fee_amount
bigint net_amount [CHECK >=0; >0 se
CREDIT_ADVANCE]
char36 pricing_policy_key
char36 risk_policy_key
timestamp expires_at
```

```
ACCOUNT ||--o{ cota
```

ACCOUNT

```
int id PK
char36 account_key UK
int customer_id FK
int status_id FK
bigint balance [CHECK >= 0]
```

```
CUSTOMER ||--o{ possui
ACCOUNT_STATUS ||--o{ estado_atual
```

IDEMPOTENCY_KEY

```
int id PK
int account_id FK
varchar scope [UNIQUE
conta+escopo+chave]
varchar idempotency_key
char64 request_hash
int response_status
jsonb response_body
```

```
ACCOUNT ||--o{ deduplica
```

BANK_SLIP_STATUS

```
int id PK
varchar enumerator UK
```

BANK_SLIP_STATUS_EVENT

```
int id PK
int bank_slip_id FK
int status_id FK
timestamp event_datetime
```

```
BANK_SLIP_STATUS ||--o{
estado_historico
BANK_SLIP ||--o{ historiza
```

CUSTOMER_DAILY_OUTGOING

```
int customer_id PK,FK
date operation_date PK
bigint consumed_amount [CHECK >= 0]
```

```
CUSTOMER ||--o{ consome
```

ACCOUNT_STATUS

```
int id PK
varchar enumerator UK
```

ACCOUNT_STATUS_EVENT

```
int id PK
int account_id FK
int status_id FK
timestamp event_datetime
```

```
ACCOUNT_STATUS ||--o{
estado_historico
ACCOUNT ||--o{ historiza
```

BILLING_PLAN

```
int id PK
char36 plan_key UK
int account_id FK
bigint base_amount
date first_due_date
bigint issuance_fee_amount
```

```
ACCOUNT ||--o{ cobra
PRICING_SNAPSHOT |o--o{ emissao
RISK_POLICY_SNAPSHOT |o--o{ decisao
```

CREDIT_ADVANCE

```
int id PK
char36 credit_advance_key UK
int account_id FK
bigint gross_amount
bigint fee_amount
bigint net_amount [CHECK =gross-fee;
>0]
```

```
ACCOUNT ||--o{ antecipa
PRICING_SNAPSHOT |o--o{ precifica
RISK_POLICY_SNAPSHOT |o--o{ decisao
```

DER 2/2 — identidade, políticas, auditoria e outbox

PK: chave interna | UK: única | FK: vínculo | relação aponta para este cartão; UUIDs são públicos

APP_USER

int id PK
char36 user_key UK
varchar email UK
varchar password_hash

PRICING_POLICY

int id PK
char36 policy_key UK
int customer_id FK
int version [UNIQUE escopo+operacao+versao]
bigint fixed_fee_cents
int percentage_basis_points
timestamp effective_from
timestamp effective_until

CUSTOMER |o--o{ personaliza

POLICY_CHANGE_REQUEST

int id PK
char36 request_key UK
int customer_id FK
int creator_user_id FK
int approver_user_id FK
varchar policy_type
jsonb payload
varchar status
char36 published_policy_key
timestamp submitted_at
timestamp approved_at

APP_USER ||--o{ cria_ou_aprova
CUSTOMER ||--o{ governa

USER_CUSTOMER_ACCESS

int id PK
int user_id FK
int customer_id FK
varchar role [UNIQUE usuario+PME]

CUSTOMER ||--o{ autoriza
APP_USER ||--o{ possui

PRICING_SNAPSHOT

int id PK
char36 policy_key
int policy_version
bigint base_amount
bigint fee_amount
timestamp applied_at

CUSTOMER |o--o{ escopo

RISK_POLICY_SNAPSHOT

int id PK
char36 policy_key
int policy_version
bigint requested_amount
bigint daily_outgoing_before
bigint daily_outgoing_after
timestamp applied_at

CUSTOMER |o--o{ escopo

OUTBOX_EVENT

int id PK
char36 event_key UK
char36 aggregate_key [sem FK de conta]
varchar topic
jsonb payload
int delivery_attempts
char36 lock_token
timestamp locked_until
timestamp next_attempt_at
timestamp published_at

USER_SESSION

int id PK
char36 session_key UK
int user_id FK
char64 refresh_token_hash UK
timestamp expires_at
timestamp revoked_at

APP_USER ||--o{ abre

RISK_POLICY

int id PK
char36 policy_key UK
int customer_id FK
int version
boolean transfer_enabled
boolean billing_plan_enabled
boolean credit_advance_enabled
bigint daily_outgoing_limit
bigint max_transfer_amount
bigint max_credit_advance_amount
int max_advance_bank_slips

CUSTOMER |o--o{ personaliza

AUDIT_EVENT

int id PK
varchar actor_type
varchar actor_key
varchar action
char36 resource_key
char64 previous_hash
char64 event_hash UK
timestamp event_datetime

Fluxos

Principal desafio

Manter saldo e ledger coerentes sob concorrência e perda de resposta exige cuidado: uma leitura sem trava pode autorizar débitos incompatíveis, e um timeout não revela se houve commit. A solução trava pelo ID interno, recarrega os dados e usa CHECK, confirmando ledger, saldo e resposta idempotente juntos. A garantia exige que toda escrita siga esse caminho; não cobre ações do administrador nem todas as falhas possíveis.

Cadastro, dinheiro e estado

O cadastro valida schema estrito, documento e duplicidade, cria o UUID e registra a transição PENDING→APPROVED. Com a conta travada, a movimentação recebe um lançamento com sinal contábil. Bloqueio e cancelamento gravam estado, evento, auditoria e outbox em um único commit do controller; CANCELLED é terminal. Duplicidade ou estado inválido retornam 409; documento ou saldo inválido, 422. A falha causa rollback sem esperar crédito futuro. Cancelar não liquida saldo nem boletos automaticamente.

Transferência, preço e risco

Após reservar a idempotência, a operação trava as contas por ID crescente com `FOR NO KEY UPDATE`, compatível com KEY SHARE da FK, e relê os dados. A política da PME prevalece sobre a padrão; a tarifa é fixo + half-up(bruto×bps/10000), com a escolha congelada em snapshot. O risco verifica habilitação e tetos por operação e por dia; um advisory lock por PME e data serializa o consumo. TRANSFER_OUT/IN/FEE compartilham operation_key, e saldo, ledger, snapshots, consumo, auditoria e resposta confirmam juntos.

Produto ou estado inválido retornam 409; saldo ou teto, 422, sem consumo parcial. Transferências A→B e B→A podem esperar circularmente sem ordenação; travar primeiro o menor ID faz uma aguardar a outra. A ordem usa ID, nunca IP. Falta de saldo é uma recusa de negócio, sem deadlock ou retry.

Regra noturna: saques e transferências acima de R$ 1.000 retornam 422 entre 20h e 6h, em America/Sao_Paulo, para qualquer conta; depósitos não são limitados. O teto diário comercial considera o principal transferido, mas a data do runtime ainda precisa ser alinhada ao fuso.

Idempotência, timeout e retry

SHA-256 canônico e UNIQUE(conta,escopo,chave) serializam reenvios. O mesmo corpo recupera o 201 original com Idempotent-Replayed; outro corpo retorna 409, e uma recusa desfaz a reserva. Só transação e antecipação repetem a operação inteira para 40P01/40001, com rollback e nova sessão, até duas tentativas totais. Não há retry para 4xx ou conector, nem espera por dinheiro.

Esgotar as tentativas retorna 503/QIT001025; timeout SQL, 503/QIT001024. Após perder a resposta, o cliente reenvia a mesma chave. As esperas são de 1s/5s no serviço externo e 2s/10s no banco, com orçamento de 15s, sem deadline global rígido.

Cobrança, reajuste e antecipação

O lote 1 emite 12 recebíveis da PME para pagadores externos, com vencimentos mensais ajustados ao fim do mês e tarifa de serviço separada. O lote 2 consulta IPCA/IGPM, trava o plano e reajusta as parcelas 13–24 com Decimal half-up, emitindo uma vez localmente. A antecipação trava a conta e os boletos próprios PENDING ainda não consumidos, soma o bruto, aplica risco e valida tarifa e líquido antes do snapshot de preço. A escolha é persistida; crédito bruto, débito da tarifa positiva e vínculo do lastro entram no commit idempotente. Antecipar não muda PENDING para PAID.

Uma falha externa retorna 502 e desfaz a escrita local, mas não a emissão externa. O lote 2 mantém o lock durante I/O, e a criação não tem replay. Lastro alheio ou inexistente retorna o mesmo 404; usado, 409. Uma lista inválida desfaz toda a tentativa. Não há empréstimo, amortização ou baixa do pagador.

Líquido positivo implementado — subparte P0.4: execução e cotação CREDIT_ADVANCE exigem bruto−tarifa > 0 após half-up. Caso contrário, retornam **422/QIT001030**, sem confirmar dinheiro, lastro, snapshots ou reserva, nem repetir a recusa de negócio. Para bruto de R$100, tarifas de R$3 e R$99,99 deixam R$97 e R$0,01; R$100 ou R$120 são recusados, preservando um saldo anterior de R$500. A tarifa de R$120 pode valer para bruto de R$1.000. O replay preserva a resposta confirmada; CHECKs e migração protegem novas gravações sem reescrever o legado. Os testes HTTP/SQL estão em COBERTURA; os demais itens 9.5 seguem pendentes.

Cotação e quatro olhos

A quote de transferência, plano ou antecipação dura 60s e persiste uma prévia, sem reservar saldo, limite, lastro ou preço. A execução recalcula e não aceita quote_key nem tarifa do cliente. Um OWNER cria DRAFT e o criador submete PENDING_APPROVAL; outro OWNER aprova sob lock e publica a versão no commit. A vigência anterior termina, sem reprecificar snapshots. Propostas pendentes não valem; autoaprovação e repetição retornam 409, e papel ou PME inválidos, 403. Rotas diretas contornam a aprovação; RETIRED não tem transição entregue. O líquido de quote TRANSFER com tarifa alta permanece uma lacuna.

Identidade e autorização

As senhas usam bcrypt, com limite de 72 bytes UTF-8. O login emite acesso de 15min e refresh opaco rotativo, cujo hash é persistido, para uma sessão de até 8h. Dispositivos têm sessões independentes e logout individual. Com JWT, VIEWER lê e cota, OPERATOR movimenta e OWNER altera estado; propostas exigem OWNER. O token interno técnico tem autoridade ampla e pode dispensar JWT nas contas. Credenciais ou JWT inválidos retornam 401; vínculo ou papel, 403. Uma proposta com sessão revogada pode retornar 403. O login não revela e-mail. Rate limit, rotação, provisionamento e gateway não foram entregues; UUID não substitui autorização.

Extrato, auditoria e outbox

O extrato ordena por data e ID; valores com sinal contábil reconstroem o saldo de uma conta estável. Lançamento alheio ou inexistente retorna o mesmo 404; offset não congela páginas diante de novas escritas. A auditoria encadeia SHA-256 sob lock global e exporta cadeia e checkpoint; o ID sequencial administrativo é uma exceção documentada. Um trigger impede UPDATE/DELETE apenas de audit_event. Ledger, estados e snapshots dependem da aplicação, e o administrador pode adulterá-los; não é blockchain.

A outbox de bloqueio e cancelamento confirma com o domínio. O worker usa claim SKIP LOCKED e lease, envia o event_key por webhook e faz ack em transações separadas. Falhas agendam backoff; uma queda após o aceite ou um lease vencido pode duplicar a entrega, exigindo deduplicação pelo consumidor. A garantia é at-least-once, sem dead-letter, envelope versionado ou exactly-once externo.

Pipelines e evidências

A CI de push/PR prepara dependências, Compose e compilação, executa guardas e a suíte completa com bootstrap isolado e preserva JUnit, métricas e metadados. O pytest cria DB, mock, portas livres e API às 21h. Uma segunda API verifica 19:59/20h, meia-noite, 05:59/6h, 12h e replay dia→noite, sem escolher relógio por HTTP ou reiniciar o desenvolvimento; o cleanup remove apenas a sessão. Bootstrap, SQL e worker não são prova HTTP. A jornada cobre quatro olhos, cobrança, reajuste, cotação, antecipação, replay, dinheiro, extrato, estado e auditoria.

Logs JSON com X-Request-ID mascaram identificadores e omitem body; tracebacks podem conter SQL/URLs. Métricas internas cobrem HTTP, QIT, replay, conectores, locks, retry, sessões e outbox, com registry por processo e gauges persistidas. Alertas são propostas, sem Prometheus, Alertmanager ou cAdvisor instalado. O benchmark de 5×40 transferências cruzadas amostra CPU/RAM como observação local, sem SLO.

Resultados, datas e comandos estão em COBERTURA, BENCHMARK e ENTREGA. CI e publicação foram confirmados em `1603118`; novas revisões exigem novo CI. Clone, aceite e ensaio do grupo seguem pendentes. O legado sample_entity está ativo, e a biblioteca não participa do produto. A RFC integral e DECISOES preservam os detalhes; os demais itens 9.5 ficam fora deste trabalho.